

Refactoring Report: Homework - 9 Visualizing Bert

Vongani Maluleke and Gemini AI Assistant

December 4, 2025

1 Introduction

This report documents the refactoring of HW9 Python code for visualizing attention in Transformer models using BertViz into an object-oriented structure using the BertVizExplorer class. The refactoring aims to improve code organization, reusability, and state management while maintaining the original flow of the homework. The core models utilized are **GPT-2** (a decoder-only model) and **BERT** (an encoder-only model), loaded via the Hugging Face **Transformers** library. Gemini was used to assist with the refactoring by prompting it to write clean and pythonic code. The code can be found here: Homework9

2 Refactoring Changes and Class Structure

The entire visualization logic has been encapsulated within the BertVizExplorer class to manage the model and tokenizer state.

2.1 Class Structure and State Management

- **Class Definition:** Introduced class BertVizExplorer to manage all subsequent operations. [3]
- **State Variables:** The model and tokenizer are now instance variables (`self.model`, `self.tokenizer`) initialized to `None` in `__init__`. This prevents global variable pollution. [3].
- **Constants:** Defined class-level constants (`MODEL_TYPE_GPT2`, `MODEL_TYPE_BERT`) for clean, non-magic string model identification. [3].

2.2 Model Loading and Visualization Methods

1. Model Loading (`load_model`)

- **Consolidated Loading:** The logic for loading both `GPT2Model` and `BertModel` was merged into a single method, `load_model(self, model_type, model_version, ...)`.
- **Type Hinting:** Added explicit type hints (e.g., `model_type: str, -> None`) for improved static analysis and readability. [1]

2. Visualization Methods

- **Neuron View (`visualize_neuron_view`):** Encapsulates the `bertviz.neuron_view.show()` function. It automatically passes the internal `self.model`, `self.tokenizer`, and `self.model_type`, simplifying external calls (e.g., `explorer.visualize_neuron_view(text_a)`).
- **Model View (`visualize_model_view`):** Encapsulates the `bertviz.model_view()` logic, including tokenization and extracting attention weights from the model output.
- **Specialized AutoModel Loading:** Introduced `load_auto_model_for_model_view()` to specifically handle the requirement of loading a model with `output_attentions=True` for the multi-head visualization, keeping it separate from the simpler `load_model` call.

Original Code (Sequential)	Refactored Code (Class-Based)	Style Improvement
<pre>model = GPT2Model... model_type = 'gpt2' def init_weights(m): ... show(model, model_type, ...)</pre>	<pre>self.model = GPT2Model... model_type=explorer.MODEL_TYPE_GPT2 except NotImplementedError: pass explorer.visualize_neuron.view(...)</pre>	Object-Oriented State Management Using Class Constants Specific Error Handling Encapsulation and Method Abstraction

Table 1: Key Refactoring Changes

2.3 Pythonic Refactoring Summary

The table below summarizes the key Pythonic changes applied to the structure:

3 Bibliography

References

- [1] Van Rossum, G., Lehtosalo, J., & Langa, L. (2015). **PEP 484 – Type Hints.** *Python Enhancement Proposal*. <https://peps.python.org/pep-0484/>
- [2] Angelico, C., Peters, T., & van Rossum, G. (2018). **PEP 572 – Assignment Expressions.** *Python Enhancement Proposal*. <https://peps.python.org/pep-0572/>
- [3] van Rossum, G. Warsaw, B., & Coghlan, A (2001). **PEP 572 – Assignment Expressions.** *Python Enhancement Proposal*. <https://peps.python.org/pep-0008/>